

Informe 4

Elaborado por:

Alejandro Zambrano (17-10684), Ángel Rodríguez (15-11669), Francisco Márquez (12-11163), Kevin Briceño (15-11661), Sergio Carrillo (14-11315).

Fecha: 07/04/2026

Resumen

El problema de satisfacibilidad booleana (SAT) es el problema de saber si una fórmula en forma normal conjuntiva tiene una asignación de variables que la haga verdadera. El problema MaxSAT es la versión de optimización donde, la fórmula insatisfacible busca ser satisfecha en el máximo de cláusulas y tiene varias especializaciones. En este estudio se estudió el problema MaxSAT simple con el *benchmark Causal Discovery* el cual contiene quince archivos de complejidad suficiente para probar distintas aproximaciones para la resolución del problema MaxSAT. Se probó una solución heurística voraz y se mejoraron las soluciones con metaheurísticas de búsqueda local: normal, iterada y tabú, recocido simulado, GRASP y un algoritmo genético. Se compararon las soluciones encontradas por los métodos propuestos con el solucionador exacto *EvalMaxSAT* y se encontró que las soluciones encontradas por los métodos planteados, a pesar de no ser óptimas en términos de satisfacción de cláusulas, son competitivas en términos de tiempo.

Palabras clave: satisfacción, heurística, metaheurística, solución exacta, óptimo.

Introducción

El problema de satisfacibilidad booleana (SAT) es el problema de saber si una fórmula en forma normal conjuntiva tiene una asignación de variables que la haga verdadera. En casos más complejos o directamente en casos de contradicciones, no se puede satisfacer la fórmula pero se busca minimizar las cláusulas que están insatisfechas: este es el problema MaxSAT. Tal problema cuenta con varias especializaciones sobre las cláusulas (que pueden tener ponderación o algún tipo de condición de obligatoriedad). En este estudio se trabajó con el problema MaxSAT puro.

Para el estudio de este problema se usó el *benchmark Causal Discovery* el cual es una transcripción de problemas de estadística para encontrar relaciones de causalidad entre sucesos correlacionados. Este *benchmark* explota al algoritmo de un solucionador por dar lugar a cláusulas densas (con muchas variables) por lo que es impráctico plantear un solucionador exacto en este estudio y se usará uno disponible en la red para comparar la solución heurística y metaheurísticas de búsqueda local, búsqueda voraz aleatorizada, recocido simulado y algoritmo genético propuestas.

Descripción del problema

El problema de satisfacibilidad booleana (SAT) es el problema de saber si dada una fórmula booleana en forma normal conjuntiva (conjunción de disyunciones), existe alguna asignación de variables tal que la fórmula sea verdadera (se satisfaga). El problema MaxSAT es un problema de optimización donde dada una fórmula SAT, que muy probablemente es insatisfacible (una contradicción) o donde muy pocas asignaciones de variables la satisfacen, se quiere una asignación de variables tal que se minimice la cantidad de cláusulas insatisfechas o, equivalentemente, se maximice las cláusulas satisfechas. [1]

En la literatura, existen varias especializaciones de MaxSAT [1]:

- MaxSAT ponderado: existen cláusulas que tienen mayor preferencia para ser satisfechas (tienen mayor peso o ponderación).
- MaxSAT parcial: existen cláusulas que obligatoriamente deben satisfacerse (*hard*) y otras que se permite que no se satisfagan (*soft*).
- MaxSAT ponderado parcial: Una combinación de los casos anteriores.

En este estudio se trabajará con el problema simple MAXSAT donde puede decirse que todas las cláusulas son *soft* y con la misma ponderación con el enfoque de minimización de cláusulas insatisfechas.

Benchmark: Descubrimiento causal

El Descubrimiento Causal (*causal discovery*) es el área de la inteligencia artificial y la estadística cuyo objetivo es inferir las relaciones de causa y efecto a partir de datos, en lugar de simplemente encontrar correlaciones. [2] Esto permite definir relaciones lógicas entre sucesos y puede traducirse al lenguaje de la lógica proposicional (dominio de SAT) con el concepto de la D-separación. [3] Cuando se trabaja con datos de muestras reales, las pruebas estadísticas de independencia suelen producir resultados contradictorios debido a la variabilidad estadística. Los métodos previos con frecuencia fallaban o se volvían ineficientes ante estas inconsistencias. [4] Mediante optimización de restricciones el problema de descubrimiento causal puede ser llevado al problema de MaxSAT. [4] Se propuso el uso de MaxSAT en [5] como el motor de búsqueda para el descubrimiento causal basado en restricciones.

Cada archivo del *benchmark* plantea una cantidad de casos para los cuales se aplica el descubrimiento causal y eso genera una cantidad de variables y cláusulas que llevan a los resolutores de MaxSAT a su límite. El resumen de los archivos se presenta en la tabla 1.

Tabla 1. Descripción de los archivos del *benchmark*

Archivo	Variables	Cláusulas	Casos	Lógica
n5	61600	221790	500 - 1k - 10k	UAI13
n6	328107	1206162	500 - 10k	UAI13
n6	12764	46236	500 - 1k - 10k	UAI14
n7	40290	145910	500 - 1k - 10k	UAI14

Solución exacta: EvalMaxSAT

EvalMaxSAT [6] es un solucionador de MaxSAT implementado en C++ y basado en el solucionador CaDiCaL [7] y el algoritmo aprendizaje un nivel a la vez (OLL por sus siglas en inglés: *One-Level-at-a-time Learning*) [8].

Solución heurística

Se propuso una heurística de ramificación estática (*Static Branching Heuristics*) basada en el conteo de literales. [9,10] La idea de la heurística se centra en tomar la variable cuya frecuencia de aparición sea máxima (regla de MOM [11]) para asignar el valor de verdad que maximice el número de cláusulas que se satisfacen en cada momento. La heurística es una simplificación de la heurística de Jerislow-Wang [12] que trabaja con esta misma idea pero con una función de costo afectada por el peso de cada cláusula.

Búsqueda local: vecindad 1-*flip*

La vecindad propuesta para cada solución consiste en alternar el valor de verdad asociado a una variable en una cláusula insatisfecha lo que implica que cada vecino de la solución difiere en exactamente una variable.

Búsqueda local iterada: perturbación y criterio de aceptación

La perturbación propuesta para esta metaheurística es la generalización *k-flip* sobre variables en cláusulas satisfechas. Esto incrementa la probabilidad de escape del cuenco de atracción que define la solución encontrada por la solución heurística propuesta al cambiar drásticamente la forma de la solución inicial. La búsqueda local se sigue haciendo con 1-*flip*.

Búsqueda tabú: movimientos tabú y criterio de aceptación

La lista tabú, donde se tienen los movimientos que no están permitidos, es un vector que guarda las iteraciones en que una variable no puede ser invertida, es decir, la variable en la posición *i*-ésima, va a ser tabú durante la cantidad de iteraciones que indique el valor del vector en el índice *i* [13].

Al comienzo, el costo de una solución se asume infinito y un movimiento se toma siempre y cuando mejore dicho costo (sea menor), por lo tanto, en la primera iteración siempre hay un movimiento a tomar. No se realizan los cambios en una iteración sino que se evalúa el costo de hacer un cambio y se elige el que tenga mayor impacto sobre la calidad de la solución si y solo si el movimiento no es tabú o es tan bueno que no es conveniente restringirlo.

La verificación de las iteraciones de restricción se verifica en $O(1)$ debido a que solamente se compara la iteración actual con el valor en un vector, sin que haya un recorrido. Dicha cantidad de iteraciones es variable con distribución uniforme. Se escoge un número entre 1 y 5 con probabilidad uniforme y eso garantiza que no se escoja restrinja siempre una variable una misma cantidad de iteraciones.

La calidad de la mejor solución global se almacena en todo momento y solamente se actualiza en caso de mejora estricta.

Recocido simulado: temperatura y enfriamiento

El enfriamiento implementado es geométrico, es decir, la temperatura decrece en un factor α cada iteración, por lo que, en la iteración n -ésima, se habrá enfriado en α^n . Antes de que se aplique el siguiente nivel de enfriamiento, se revisa por cierta cantidad de iteraciones, todos los vecinos posibles a esa temperatura, pero a diferencia de los vecinos explorados con la búsqueda tabú, aquí el vecino es seleccionado al azar. El criterio de aceptación viene por la probabilidad similar a la distribución de Boltzmann y si una solución candidata es mejor que lo que se tiene, se toma sin miramientos. Si no, la probabilidad de tomarla varía con la temperatura [14].

GRASP: la lista restringida de mejores candidatos

La función de costo consiste en seleccionar el máximo entre la cantidad de veces que una variable está negada o no (del mismo modo que en la solución heurística propuesta previamente). El umbral para entrar en la lista restringida de mejores candidatos se calcula con un factor α de 0,2. Esto satisface la condición voraz y aleatorizada [15].

La adaptatividad del algoritmo viene porque una vez seleccionada una parte de la solución, se actualizan las frecuencias asociadas a esa inclusión y entonces serán consideradas en la siguiente iteración.

Algoritmo genético

Para el algoritmo genético ya se tienen implementados el genotipo y el fenotipo de la población desde la solución heurística: el genotipo es la representación como vector de valores de las variables, donde el índice es la variable y el fenotipo es la representación global de la asignación como cadena binaria. La función de aptitud también está considerada y es justamente la cantidad de cláusulas insatisfechas [16, 17].

Se implementaron los operadores de selección, cruce y mutación. El operador de selección de los padres es un torneo donde compiten tres candidatos de una generación. El operador de cruce es un cruce uniforme, es decir, para cada una de las n variables se escoge con igual probabilidad al padre A o al B y así se construye una nueva solución que no es demasiado parecida a ninguno de los dos padres. Luego, si hay n variables, la probabilidad de que una variable mute (invierta su valor) es $1/n$ [18]. Se probó con cincuenta individuos y cien generaciones.

Algoritmo memético

Dado que el algoritmo memético es un algoritmo genético con algún proceso de mejora de los descendientes, se modificó el algoritmo genético para que las soluciones hija realizaran una búsqueda local manteniendo el cruce [19, 20] y la mutación [18] y cambiando la selección a un torneo de dos candidatos, así, se evita la convergencia temprana a óptimos locales y se permite la exploración de otras regiones del espacio de búsqueda [19]. Se probó con veinte individuos [21, 22] y las generaciones fueron dinámicas (por tiempo de ejecución y veinticinco generaciones sin mejora).

Búsqueda dispersa y reenlazado de caminos

Para la búsqueda dispersa se usó un conjunto de referencia de diez soluciones con cinco soluciones prometedoras (bajo costo) y cinco soluciones diversas (más distancia de Hamming) [23, 24]. Para el reenlazado de caminos se implementó un reenlazado bidireccional [25] truncado con mejora local [26, 27]. Dado que la topología de la vecindad para MAXSAT es asimétrica, el reenlazado de caminos bidireccional permite explorar regiones distintas a partir de las mismas soluciones guía. Por su parte el truncado con mejora local intensifica sobre la mejor solución intermedia vista hasta el momento.

Optimización de colonia de hormigas

Se implementó una optimización de colonia de hormigas híbrida donde cada hormiga realiza una búsqueda local sobre las soluciones que encuentra y se usaron diez hormigas para esto [28]. El peso de las feromonas fue 1,0 para garantizar una influencia lineal sobre las decisiones que toma cada hormiga [29, 30]. El peso de la heurística fue 2,0 para una influencia cuadrática sobre las decisiones, lo que implica que una hormiga debe considerar más la calidad de las soluciones que la memoria de lo que ya se ha visto [29, 30]. Está demostrado que, con estas condiciones, se obtienen resultados competitivos [31].

Optimización de reacciones químicas

Como metaheurística propia se escogió, por motivos de tiempo y evitar redundancia con la literatura, la optimización de reacciones químicas [32]. En las siguientes líneas se explicará brevemente la inspiración y elementos de la metaheurística así como los operadores involucrados en la misma.

Inspiración

Las reacciones químicas ocurren porque las moléculas que en ellas participan *perciben* que existe un estado en el cual su estabilidad es mayor. Se puede decir que cada molécula nota este cambio por su energía potencial (U). La energía potencial es, *a grosso modo*, la cuantificación de los efectos internos de la molécula (tensiones estéricas y electrónicas) y básicamente, la estabilidad de una molécula es inversamente proporcional a U [33, 34, 35]. Se ha teorizado que las reacciones ocurren porque las moléculas chocan entre sí y cuando las colisiones son efectivas, la transformación ocurre (teoría cinético molecular, TCM) [33, 34, 35]. La metaheurística parte del postulado de la TCM de las colisiones y se enfoca en las siguientes reacciones (la explicación a detalle de cada tipo de reacción en la naturaleza escapa del alcance de este informe):

1. Síntesis (colisiones efectivas entre dos moléculas): por ejemplo (en el sentido químico) la reacción entre los protones y los aniones hidróxido para formar agua (aunque más correctamente son los iones hidronio porque no es posible aislar protones sino hidronio, justamente, por efectos electrónicos) y en general, reacciones bimoleculares. Existen reacciones trimoleculares, pero son muy poco comunes por la baja probabilidad de que tres moléculas colisionen al mismo tiempo y la colisión sea efectiva. Un ejemplo es la combinación de dos moléculas de óxido nítrico (NO) con cloro gaseoso para formar dos moléculas de cloruro de nitrosilo.
2. Descomposición: reacciones que ocurren con moléculas con alta energía (U) y que dan como resultado al menos dos moléculas más pequeñas, por ejemplo, la reacción química como la dismutación del agua oxigenada en agua y oxígeno debido a la elevada inestabilidad del enlace O-O del peróxido que no deja de ocurrir espontáneamente a ninguna temperatura. También, las descomposiciones pueden ocurrir por efecto del entorno sobre moléculas que son suficientemente estables para no descomponerse espontáneamente, por ejemplo, dejar bromo líquido a la luz hará que desaparezca su color rojo característico por la disociación del bromo molecular en dos radicales bromo. Este es un efecto que también lo puede proporcionar la temperatura (que es la manera directa de hablar de energía cinética, K).
3. Colisiones *inefectivas*: son colisiones que no tienen la energía necesaria para inducir una reacción química (convertir una molécula A en una B totalmente diferente), pero sí pueden inducir otros cambios. En la naturaleza esto puede ilustrarse con el tautomerismo ceto-enol que puede exhibir la acetona que es, básicamente, cambiar de posición algunos enlaces químicos.

Todos estos cambios ocurren porque, “para la molécula”, el estado de los productos es más estable que el estado de los reactivos porque se minimiza su energía interna.

Operadores y estructuras de la metaheurística.

La metaheurística modela los efectos naturales definidos previamente con un búfer de energía (el entorno), cuya función es la de almacenar energía que se libera de una reacción luego de que ocurre, o entregarle energía a una reacción para que ocurra. Las soluciones factibles de los problemas de optimización son las moléculas. Estas tienen atributos como la estructura de la solución que es altamente dependiente del problema y en el caso de MaxSAT es el arreglo de bits planteado desde el comienzo y la energía cinética (K) que tiene la molécula, es decir, una medida de su temperatura (en el mismo sentido que en recocido simulado) y también puede tener su energía potencial (que es el valor de la función de evaluación) y los algoritmos de síntesis, descomposición y colisiones inefectivas. Asimismo,

cada molécula tiene un contador de colisiones inefectivas que ha sufrido y que sirven como un nivel de energía interna que, cuando se sobrepasa, la molécula solamente puede sufrir una reacción de descomposición.

La forma de saber si una reacción química ocurre, o no, es con la regla:

$$E_{inicial} \geq U_{final}$$

Donde $E_{inicial} = U_{inicial} + K_{inicial} + buffer$ y $buffer$ es opcional ya que solamente actúa cuando no se cumple la condición mencionada. Además, si dicha regla falla, la reacción no puede ocurrir. Asimismo, la energía se pierde por colisión con una tasa q y la energía que pasa al búfer es $q \times K$ en caso de colisión inefectiva y $q \times (E_{inicial} - U_{final})$ siempre que una reacción ocurra.

Para los efectos de este estudio, se determinó que las condiciones que otorgaban la mejor calidad de resultados eran con un tiempo límite de dos segundos, límite de colisiones inefectivas de diez y energía cinética inicial de mil y pérdida de energía por colisión de veinte por ciento. También se mantuvo con una colección de veinte moléculas.

Resultados de corridas

En la tabla 2 se muestran los resultados en número de cláusulas insatisfechas promedio de treinta ejecuciones de cada archivo del *benchmark*. El número entre paréntesis corresponde a la desviación estándar sobre la última cifra significativa. Si no tiene número entre paréntesis es porque la calidad de la solución no varío con las repeticiones. Los valores vacíos son resultados que no se pudieron obtener a tiempo para la fecha de entrega por el propio rendimiento del algoritmo frente a la instancia.

Tabla 2. Calidad de las soluciones por algoritmo

Archivo	Casos	Exacto	Heurística	B. L.	B. L. I.	B. T.	R. S.	GRASPA. G.	A. M.	B. D.	C. H.	O. R. Q.	
n5 i2	500	5	1952	223	195	1512	1952	185	1905	37	222	141	1952
n5 i4	500	4	1949	219	186	1509	1949	181	1887	40	219	126	1949
n5 i5	10k	5	1953	222	182	1513	1953	193	1908	37	223	131	1953
n5 i7	1k	10	1972	242	208	1532	1972	192	1923	60	242	158	1972
n5 i8	10k	5	1977	53	52	1537	1977	203	1923	69	247	154	1977
n6 i1	500	86	7672	623	544	6402	7672	533	7652	363	623	431	7672
n6 i4	500	200	56	152	150	51	56	53	56	49	53	51	56
n6 i5	10k	222	7648	599	538	6378	7648	509	7618	149	599	443	7648
n6 i7	1k	8	155	141	140	150	155	152	154	461	152	151	155
n6 i8	1k	8	144	66	65	139	144	141	144	139	141	139	144
n6 i9	10k	187	69	247	207	64	69	66	68	63	66	64	69
n6 i9	1k	187	69	66	64	64	69	66	69	63	66	64	69
n7 i8	10k	20	378	372	374	370	378	372	378	370	372	370	378
n7 i8	1k	20	378	372	373	370	378	372	378	368	372	370	378
n7 i9	500	474	220	214	216	212	220	214	220	211	214	212	220

En la tabla 3 se puede observar el tiempo en segundos que le tomó a cada algoritmo con cada instancia del *benchmark*. El número entre paréntesis es la desviación estándar sobre la última cifra significativa. Los valores vacíos son resultados que no se pudieron obtener a tiempo para la fecha de entrega por el propio rendimiento del algoritmo frente a la instancia. Se puede observar que la implementación de diferentes algoritmos lleva a distintos tiempos de ejecución, también la diferencia con los tiempos se debe a que la estructura de las cláusulas es muy densa [5] en estos problemas (cada cláusula puede tener suficientes variables para actuar como un cuello de botella). Asimismo, para esta entrega los distintos algoritmos fueron ejecutados en distintas máquinas lo que implica distinto poder de cómputo. Sin embargo, es notable que para la mayor parte de las heurísticas y metaheurísticas implementadas, si bien la calidad de la solución no es globalmente óptima, la diferencia en tiempo para obtenerlas plantea una ventana de oportunidad para estos algoritmos siempre y cuando no sea imprescindible una solución exacta del problema.

Con la metaheurística seleccionada, puede observarse que no mejora la calidad de la solución heurística y esto pudiera deberse, como en el caso de sus homólogas que al tomar como parte de las soluciones a la entregada por la heurística constructiva voraz, la metaheurística no parece encontrar mejores soluciones en las adyacencias de la solución heurística dentro del espacio de búsqueda.

Tabla 3. Tiempo de ejecución por algoritmo

Archivo	Casos	Exacto	Heurística	B. L.			A.			B.		O. R.	
				B. L.	I.	B. T.	R. S.	GRASP	G.	A. M.	D.	C. H.	Q.
n5 i2	500	9(2)e2	3,5(2)	0,04(2)	1,4(2)	1,4(2)	0,008(1)	500(1)	27(3)	60,6(4)	22(4)	32(4)	2.00009(5)
n5 i4	500	9(2)e2	3,4(2)	0,04(2)	1,4(2)	1,5(3)	0,009(2)	500(1)	29(2)	60,5(4)	21(6)	33(6)	2.00009(5)
n5 i5	10k	1.1(2)e3	3,4(2)	0,04(2)	1,5(3)	1,4(2)	0,011(5)	500(1)	29(2)	60,6(3)	22(5)	34(4)	2.00008(6)
n5 i7	1k	1.5(3)e3	3,4(2)	0,05(1)	1,5(3)	1,5(1)	0,009(2)	510(9)	27(1)	60,6(4)	21(5)	35(6)	2.00010(5)
n5 i8	10k	1.5(1)e2	3,4(2)	0,004(3)	0,3(1)	1,5(1)	0,009(2)	500(1)	27(2)	60,8(5)	23(6)	32(5)	2.00009(6)
n6 i1	500	1800.8(3)	113,63416	0,2(3)	4,2(9)	7(3)	0,013(6)	10879	80(3)	65(4)	59(5)	60,12(9)	2.0009(1)
n6 i4	500	1800.10(4)	10,17(1)	0,006(4)	0,3(1)	0,4(1)	0,003(1)	10(1)	3,3(8)	16(1)	6(2)	3,7(5)	2.00004(2)
n6 i5	10k	7(3)e2	113,78656	0,1(1)	3,5(5)	7(2)	0,011(3)	10977	80(3)	16(2)	58(4)	60,11(5)	2.0010(1)
n6 i7	1k	119(2)	0,17(2)	0,006(4)	0,34(9)	0,5(1)	0,004(1)	10(1)	3,5(7)	65(4)	7(2)	4,9(7)	2.00004(3)
n6 i8	1k	104(1)	0,16(1)	0,006(3)	0,32(8)	0,5(1)	0,004(1)	10(1)	3,5(7)	16(2)	6(2)	4,5(6)	2.00005(3)
n6 i9	10k	1.8(1)e3	0,17(2)	0,04(2)	1,3(3)	0,4(2)	0,003(1)	10(2)	3,5(7)	17(3)	6(2)	3,9(7)	2.00005(2)
n6 i9	1k	1800.12(8)	0,17(1)	0,004(2)	0,29(9)	0,4(1)	0,004(1)	10(1)	3,5(8)	17(2)	6(3)	4,1(7)	2.00005(2)
n7 i8	10k	2.7(2)e2	1,54(7)	0,02(1)	1,1(2)	1,5(2)	0,008(3)	170(2)	21(2)	58(3)	21(4)	23(3)	2.00008(6)
n7 i8	1k	263(7)	1,6(1)	0,02(1)	1,1(2)	1,6(2)	0,008(2)	170(3)	22(2)	57(5)	21(5)	22(3)	2.00007(4)
n7 i9	500	1800.2(2)	1,57(9)	0,0120(7)	0,69(2)	1,6(2)	0,007(2)	180(3)	21(2)	50(5)	20(6)	19(3)	2.00008(5)

Asimismo, el incremento en la cantidad de algoritmos implementados y los parámetros a controlar para cada uno, sumado a la necesidad estadística de tener réplicas, plantea un cuello de botella ineludible en la cantidad de pruebas que se pueden realizar frente a los tiempos de entrega de los distintos avances.

Debido a los resultados de calidad de la solución, se probó con tiempos límites de dos y sesenta segundos lo que no resultó en mejoras para la solución heurística así que se reportaron los valores con dos segundos.

Conclusiones

Las soluciones encontradas por las heurísticas y metaheurísticas planteadas, a pesar de no competir frente a un solucionador exacto en términos de calidad de la solución, compiten muy bien en términos de rendimiento temporal.

Se requieren más ensayos de tiempos de ejecución de estos experimentos para encontrar las condiciones que generen el mejor balance entre tiempo de ejecución y la calidad de las soluciones.

Para el problema de optimización SAT, la metaheurística que entregó la mejor calidad de soluciones fue el algoritmo memético y las que no mejoraron a la solución voraz constructiva fueron recocido simulado y optimización de reacciones químicas.

Referencias

- [1] da Silva, P. F. M. (2010). *Max-SAT algorithms for real world instances*. (Master Dissertation).

- [2] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.
- [3] Hyttinen, A., Hoyer, P. O., Eberhardt, F., & Jarvisalo, M. (2013). *Discovering cyclic causal models with latent variables: A general SAT-based procedure*. arXiv preprint arXiv:1309.6836.
- [4] Hyttinen, A., Eberhardt, F., & Jarvisalo, M. (2014, July). *Constraint-based Causal Discovery: Conflict Resolution with Answer Set Programming*. In UAI (pp. 340-349).
- [5] Berg, O. J., Hyttinen, A. J., & Jarvisalo, M. J. (2019). *Applications of MaxSAT in data analysis*. In International Conferences on Theory and Applications of Satisfiability Testing (pp. 50-64). EasyChair Publications.
- [6] Avellaneda, F. (2020). *A short description of the solver EvalMaxSAT*. MaxSAT Evaluation, 8, 364.
- [7] A. Biere, K. Fazekas, M. Fleury and M. Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020*.
- [8] A. Morgado, C. Dodaro, and J. Marques-Silva, *Core-guided MaxSAT with soft cardinality constraints*.
- [9] Silva, J. M., & Sakallah, K. A. (1996). *GRASP-a new search algorithm for satisfiability*. In Proceedings of International Conference on Computer Aided Design (pp. 220-227). IEEE.
- [10] Liang, J. H., Ganesh, V., Poupart, P., & Czarnecki, K. (2016, June). *Learning rate based branching heuristic for SAT solvers*. In International Conference on Theory and Applications of Satisfiability Testing (pp. 123-140). Cham: Springer International Publishing.
- [11] Dubois, O., André, P., Boufkhad, Y., & Carlier, J. (1996). *Sat versus unsat*. Johnson and Trick, Second DIMACS Series in Discrete Mathematics and Theoretical Computer Science, American Mathematical Society, 415-436.
- [12] Jeroslow, R. G., & Wang, J. (1990). *Solving propositional satisfiability problems*. Annals of mathematics and Artificial Intelligence, 1(1), 167-187.
- [13] Taillard, É. (1991). *Robust taboo search for the quadratic assignment problem*. Parallel computing, 17(4-5), 443-455.
- [14] Johnson, D. S., Aragon, C. R., McGeoch, L. A., & Schevon, C. (1989). *Optimization by simulated annealing: An experimental evaluation; part I, graph partitioning*. Operations research, 37(6), 865-892.
- [15] Feo, T. A., & Resende, M. G. (1995). *Greedy randomized adaptive search procedures*. Journal of global optimization, 6(2), 109-133.
- [16] Bäck, T. (1993). *Optimal mutation rates in genetic search*. In Proceedings of the 5th international conference on genetic algorithms (pp. 2-8).
- [17] Rana, S., & Whitley, D. (1998). *Genetic algorithm behavior in the MAXSAT domain*. In International Conference on Parallel Problem Solving from Nature (pp. 785-794). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [18] Bäck, T. (1993). *Optimal mutation rates in genetic search*. In Proceedings of the 5th international conference on genetic algorithms (pp. 2-8).
- [19] Gottlieb, J., Marchiori, E., & Rossi, C. (2002). *Evolutionary algorithms for the satisfiability problem*. Evolutionary computation, 10(1), 35-50.
- [20] Eiben, A. E., van Kemenade, C. H., & Kok, J. N. (1995, June). *Orgy in the computer: Multi-parent reproduction in genetic algorithms*. In European conference on artificial life (pp. 934-945). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [21] Goldberg, D. E., & Deb, K. (1991). *A comparative analysis of selection schemes used in genetic algorithms*. In Foundations of genetic algorithms (Vol. 1, pp. 69-93). Elsevier.
- [22] Moscato, P. (1989). *On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms*. Caltech concurrent computation program, C3P Report, 826(1989), 37.
- [23] Neri, F., Cotta, C., & Moscato, P. (Eds.). (2011). *Handbook of memetic algorithms*. Springer.
- [24] Rego, C. (2006). *Scatter Search: Methodology and Implementations in C*.
- [25] Glover, F., Laguna, M., & Martí, R. (2000). *Fundamentals of scatter search and path relinking*. Control and cybernetics, 29(3), 653-684.

- [26] Resendel, M. G., & Ribeiro, C. C. (2005). *GRASP with path-relinking: Recent advances and applications*. Metaheuristics: progress as real problem solvers, 29-63.
- [27] Glover, F. (1997). *A template for scatter search and path relinking*. In European conference on artificial evolution (pp. 13-54). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [28] Resende, M. G., & Ribeiro, C. C. (2013). *GRASP: Greedy randomized adaptive search procedures*. In Search methodologies: introductory tutorials in optimization and decision support techniques (pp. 287-312). Boston, MA: Springer US.
- [29] Stützle, T., & Hoos, H. H. (2000). *MAX-MIN ant system*. Future generation computer systems, 16(8), 889-914.
- [30] Dorigo, M. (2007). *Ant colony optimization*. Scholarpedia, 2(3), 1461.
- [31] Dorigo, M., Maniezzo, V., & Coloni, A. (1996). *Ant system: optimization by a colony of cooperating agents*. IEEE transactions on systems, man, and cybernetics, part b (cybernetics), 26(1), 29-41.
- [32] A. Y. S. Lam and V. O. K. Li, *Chemical-reaction-inspired metaheuristic for optimization* IEEE Trans. Evol. Comput., vol. 14, no. 3, pp. 381–399, Jun. 2010.
- [33] P. Atkins and J. de Paula, *Physical Chemistry*, 10th ed. Oxford, U.K.: Oxford Univ. Press, 2014.
- [34] G. W. Castellan, *Physical Chemistry*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1983.
- [35] I. N. Levine, *Physical Chemistry*, 6th ed. New York, NY, USA: McGraw-Hill, 2008.